

Rapporto di Analisi Critica del Codice

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.date.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
23
```

1. Capire cosa fa il programma senza eseguirlo.
2. Individuare nel codice sorgente le casistiche non standard che il programma non gestisce (esempio, comportamenti potenziali che non sono stati contemplati).
3. Individuare eventuali errori di sintassi / logici.
4. Proporre una soluzione per ognuno di essi.

1. Cosa fa il programma (Analisi Statica)

Il programma implementa un semplice assistente virtuale interattivo a riga di comando. All'interno di un ciclo continuo (`while True`), il programma richiede ripetutamente un input testuale all'utente. Se l'utente scrive "esci", il programma saluta e interrompe l'esecuzione. Per qualsiasi altra stringa inserita, invoca la funzione `assistente_virtuale(comando)` , la quale confronta l'input con tre domande predefinite ("Qual è la data di oggi?", "Che ore sono?", "Come ti chiami?"). Se la domanda corrisponde, restituisce una risposta dinamica (usando il modulo `datetime`) o statica; altrimenti, restituisce un messaggio di errore predefinito.

2. Errori di Sintassi e Logici Rilevati

- **Righe 3-9 VS 11: Errore di Forward Reference (Logico/Sintattico).** In Python il codice viene letto dall'alto verso il basso. Il ciclo `while` chiama la funzione `assistente_virtuale` alla riga 9, ma quest'ultima viene definita solo alla riga 11. Questo genera un `NameError` immediato.
La soluzione a questo problema sta nel spostare l'intero blocco della funzione (righe 11-22) all'inizio del codice
- **Riga 3: Errore Sintattico.** nella funzione while bisogna mettere i ":" e quindi la riga corretta sarebbe "while True:"
- **Riga 7: Errore di indentazione.** Il `break` alla riga 7 è allineato verticalmente con l'`if` della riga 5, non con il `print` della riga 6. In Python l'indentazione definisce i blocchi di codice. Se il `break` doveva far parte dell'azione legata all'istruzione `if`, doveva essere rientrato (indentato) esattamente come il `print`
- **Riga 13: Errore di Attributo (AttributeError).** La funzione `datetime.datetoday()` non esiste nel modulo standard. La sintassi corretta è `datetime.date.today()`.
- **Riga 16-17: Errore di Tipo (TypeError).** Chiamando `.time()` alla riga 16, la variabile diventa un oggetto `datetime.time`. Gli oggetti di questo tipo non supportano il metodo `.strftime()` invocato alla riga successiva, causando un crash.

3. Casistiche Non Standard e Comportamenti Non Gestiti (Pensiero Critico/Hacker)

- **Sensibilità al Case (Case Sensitivity):** Se l'utente scrive "Esci", "CHE ORE SONO?" o "qual è la data di oggi?" (con l'iniziale minuscola), il programma fallisce nel riconoscere i comandi a causa del confronto rigido `==`.
- **Spazi Vuoti (Whitespace):** Inserendo accidentalmente uno spazio prima o dopo la frase (es. " esci" o "Che ore sono? "), l'uguaglianza stringa salta e il comando viene ignorato o rifiutato.
- **Mancanza di Sanificazione:** Il programma non pulisce né normalizza l'input, rendendo l'interazione estremamente fragile a imprevisti o input sporchi.

4. Soluzione e Codice Corretto

Di seguito viene proposta la versione corretta e ottimizzata del programma, che risolve tutti gli errori di runtime e implementa una sanificazione basilare dell'input tramite `.strip()` e una flessibilità sui caratteri con `.lower()` (adeguando i controlli):

```
import datetime

def assistente_virtuale(comando):
    comando_normalizzato = comando.strip().lower()

    if comando_normalizzato == "qual è la data di oggi?":
        oggi = datetime.date.today()
        risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
    elif comando_normalizzato == "che ore sono?":
        ora_attuale = datetime.datetime.now()
        risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
    elif comando_normalizzato == "come ti chiami?":
        risposta = "Mi chiamo Assistente Virtuale"
    else:
        risposta = "Non ho capito la tua domanda."

while True:
    comando_utente = input("Cosa vuoi sapere? ")

    if comando_utente.strip().lower() == "esci":
        print("Arrivederci!")
        break
    else:
        print(assistente_virtuale(comando_utente))
```